HW 2: Advanced Behavior Cloning

Eli Yale
University of Utah
Course: ROBOT 6960
Instructor: Daniel Brown
Date: January 26, 2026

Introduction:

In this report I summarize my efforts to create a new 2d robot environment and learning task from scratch. The environment is designed to simulate an underwater ROV that is tasked to avoid moving obstacles by using only a sonar sensor. I record my expert human demos via a manual keyboard input mode. Then I apply three imitation learning policies to the problem: a random policy, a policy based on standard behavior cloning, and a policy based on the Action Chunking Transformer proposed by (Zhao et al., 2023). I evaluate the three policies based on how long the model survives in the environment without a collision. I was able to get standard behavior cloning to perform the best while finding interesting emergent behaviors in my ACT implementation.

Environment:

The environment simulates an underwater AUV navigating around moving obstacles like fish or other ROVs. The agent has a simulated sonar sensor with 17 rays spanning 180° from its nose. At each step, it observes 17 ray distances, linear acceleration, angular velocity, and heading (20 inputs total) and outputs continuous steering and acceleration. A raycasting algorithm returns either the max sonar range or a shorter distance if an obstacle is detected. The agent and obstacles are modeled as circles for simplified orientation and collision detection, while boundaries do not count as obstacles. A manual mode allows a human expert to control the AUV in real time, recording observations and actions for imitation learning. Figure 1 shows a screenshot of the environment implemented with PyGame.

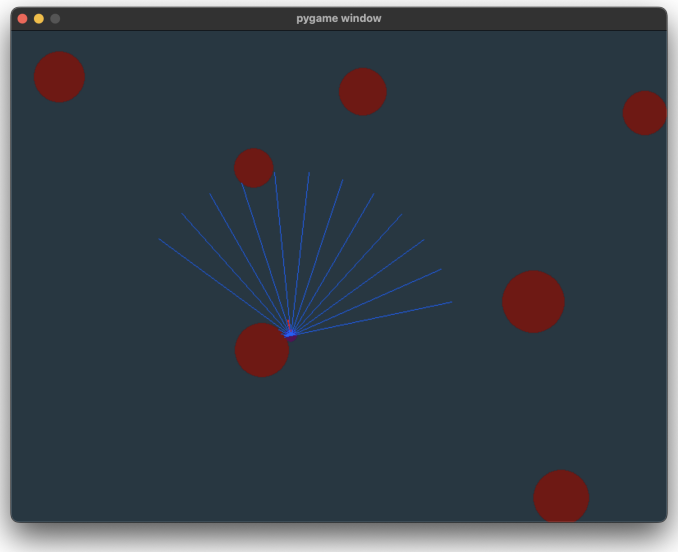


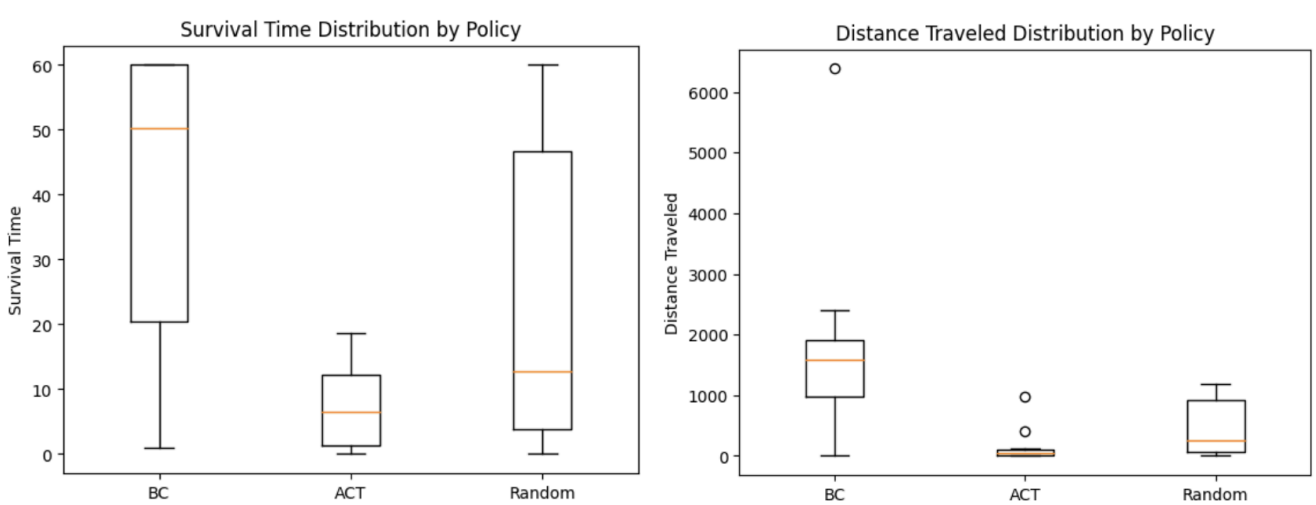
Figure 1: The acoustic world showing the agent and heading, sonar rays, and obstacles.

Policies

1. I implemented a simple random policy that accelerates to a constant velocity then randomly picks a steering angle.
2. Next, I adapted the standard behavior cloning neural network from the mountain car problem. I changed the input and output dimensions to match this problem space. I also switched from a classifier to a regressor with mean squared error loss since my action space is continuous. There is one fully connected hidden layer of size 256.
3. Finally, I attempted to implement the Action Chunking Transformer as described by (Zhao et al., 2023). My theory was that chunked actions may help the agent better avoid obstacles by capturing temporal structure and coupling between the sonar sensor measurements and inertial state. Furthermore, this problem is multimodal; there are several ways to avoid an obstacle such as turning, accelerating, or staying still and the expert demos should capture these complexities. Lastly, the low dimensional sonar sensor should be easier to train on a personal computer compared to a camera feed. I got a bit lost on the implementation and leaned heavily on AI. However I was able to get a working model that could train on my chunked expert dataset, had decreasing loss over training epochs, and was able to generate an action sequence given an observation. My version was more of an ACT-lite that didn't have the latent style variable.

Results

I trained both policies on one single demonstration that lasted about 5 mins. The dataset was split and chunked for ACT. I got good performance with BC compared to the random policy while ACT was never successful in achieving full 60 second survival. For each policy, I ran 10 simulations capped at 60 seconds and I recorded the survival times and total distance traveled. The boxplots for these results are shown in Figure 2. Standard BC was able to survive the full 60 seconds often while traveling the farthest to evade the obstacle. The random policy shows the expected large variance in survival time. ACT has the lowest variance and most consistent policy indicating it was taking consistent actions, just not correct ones.



I also subjectively analyzed several simulation runs for each policy by rendering them and watching in real time. The ACT learned interesting behaviors including spinning rapidly and backing away. I observed it make several rapic evasions in tight conditions. For a reason I can't explain, ACT performed

subjectively better in my rendered simulations than in the headless versions I ran and reported above. Its motion was generally very fast which could be due to an implementation error. Standard BC performed incredibly well and learned many of my expert tactics including turning away and changing acceleration.

Lessons Learned

The most challenging part of this experiment was understanding the intricate details of the ACT architecture. A model like ACT is a huge step up from vanilla BC. I was working with many more hyper parameters, a more complex dataset, and PyTorch primitives that I was unfamiliar with. Chunking my dataset ended up being challenging and I think I should have collected many more demos to get a larger dataset covering more scenarios. I also spent lots of time playing around with normalizing the actions and observations then denormalizing for the environment. I gained a deeper understanding of how chunks work and how to go from the output sequences to a single action. AI was very helpful in answering my question, solving bugs and often giving me full methods as long as I was guiding it in the right direction. The easiest part was setting up the PyGame environment and overall I was quite successful in getting things to work. I think with a bit more in depth analysis of the ACT hyperparameters and training pipeline I could get it to behave soundly in my custom acoustic world.

Future work:

I would like to experiment with different evaluation metrics. My initial metric based only on survival time is a bit tricky to evaluate in large batches and subject to discrimination based on the random initial state. For example if on environment creation, one obstacle has a high initial velocity straight towards the agent, then the agent has little chance of avoiding it. To mitigate this I could tweak the evaluation metric to something like the number of collisions in a 60 second simulation window. This wouldn't penalize the policy for an early collision and instead give it a chance to recover and continue.
I would also like to make the simulated underwater environment richer by adding things like currents, noise interference, and stationary obstacles that can be used for hiding. Also, the boundary should be added as an obstacle. I have uploaded all my code on my github account at https://github.com/eliyale/acoustic_bc and plan to keep working on it.

References:

Zhao, T., Xie, A., Wu, H., Zeng, A., He, B., Tang, Y., Liu, M., Zhang, L., & Yu, T. (2023). *Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware*. arXiv preprint arXiv:2304.13705.